

BÀI 13. PHÁT TRIỂN WEBSITE VỚI DJANGO - PHẦN 3 (TEMPLATES & FORMS CƠ BẢN)

13.1. Django Templates

13.1.1. Khái niệm Template: file HTML với các thẻ đặc biệt của Django

Template trong Django là các file văn bản (thường là HTML) chứa nội dung tĩnh cùng với các cú pháp đặc biệt của **Django Template Language (DTL)**. Mục đích của Template là tách biệt hoàn toàn phần giao diện người dùng (presentation) khỏi logic nghiệp vụ (business logic) của ứng dụng.

- Khi một View nhận được yêu cầu và xử lý xong dữ liệu, thay vì trả về `HttpResponse` với một chuỗi HTML "cứng", View sẽ chọn một Template, "đổ" dữ liệu vào Template đó, và sau đó trả về HTML đã được render cho trình duyệt.
- Template cho phép tái sử dụng mã HTML và tạo ra giao diện động dựa trên dữ liệu từ Python.

13.1.2. Ngôn ngữ Template của Django

DTL có hai loại cú pháp chính:

1. **Biến (Variables):** Dùng để hiển thị dữ liệu được truyền từ View.

- **Cú pháp:** `{{ variable_name }}`
- Django sẽ thay thế `{{ variable_name }}` bằng giá trị của biến đó.
- Bạn có thể truy cập thuộc tính hoặc phần tử của biến bằng dấu chấm: `{{ object.attribute }}`, `{{ dictionary.key }}`, `{{ list.0 }}`.

Ví dụ:

```
<h1>{{ post.title }}</h1>
<p>{{ post.content }}</p>
<small>Đăng ngày: {{ post.pub_date }}</small>
```

2. **Thẻ (Tags):** Dùng để thực hiện logic trong Template, như vòng lặp, điều kiện, kế thừa, v.v.

- **Cú pháp:** `{% tag_name arguments %}`
- Thẻ thường có thẻ đóng: `{% tag_name %} ... {% endtag_name %}`.

Ví dụ:

```
{% if user.is_authenticated %}
    <p>Chào mừng, {{ user.username }}!</p>
{% else %}
    <p>Bạn chưa đăng nhập.</p>
{% endif %}
{% for item in items %}
    <li>{{ item.name }}</li>
{% endfor %}
```

3. Bình luận (Comments):

- `{# This is a single-line template comment #}`: Bình luận sẽ không xuất hiện trong mã nguồn HTML cuối cùng.
- `{% comment %}` ... `{% endcomment %}`: Bình luận đa dòng.

13.1.3. Cấu hình thư mục Templates trong settings.py

Django cần biết nơi tìm các file Template của bạn. Bạn cấu hình điều này trong biến `TEMPLATES` trong file `settings.py` của project.

Mặc định, Django sẽ tìm kiếm các template trong thư mục `con templates` của mỗi App đã đăng ký. Tuy nhiên, bạn cũng có thể định nghĩa các thư mục template "tổng thể" cho toàn bộ project.

```
# myportfolio/settings.py
import os # Đảm bảo dòng này có ở đầu file
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [os.path.join(BASE_DIR, 'templates')], # Thêm dòng này
        'APP_DIRS': True, # Cho phép Django tìm kiếm templates trong thư
        mục 'templates' của mỗi app
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    ],
]
```

- **'DIRS': [os.path.join(BASE_DIR, 'templates')]**: Điều này chỉ định rằng Django cũng nên tìm kiếm các template trong thư mục templates nằm ở cấp độ gốc của project của bạn. Bạn nên tạo thư mục này:

```
myportfolio/  
├-- venv/  
├-- myportfolio/  
|   ├── ...  
|   └-- templates/      # Thư mục template cấp project  
|       └-- base.html  
├-- blog/  
|   ├── templates/      # Thư mục template cấp app  
|   |   └-- blog/  
|   |       └-- post_list.html  
|   └-- ...
```

Thông thường, các template của từng App nên được đặt trong app_name/templates/app_name/ để tránh xung đột tên nếu có nhiều App cùng có template tên index.html.

13.1.4. Cách truyền dữ liệu từ View sang Template

Để truyền dữ liệu từ một View sang một Template, bạn sử dụng hàm render().

Hàm render():

- **Cú pháp:** render(request, template_name, context=None)
 - request: Đối tượng HttpRequest nhận được từ người dùng.
 - template_name: Tên của file template cần render (ví dụ: 'blog/post_list.html').
 - context: (Tùy chọn) Một Dictionary mà các khóa của nó sẽ trở thành tên biến trong template, và các giá trị của chúng là dữ liệu bạn muốn truyền.

Ví dụ:

```
# blog/views.py  
  
from django.shortcuts import render # Import hàm render  
from .models import Post  
  
def post_list(request):  
    all_posts = Post.objects.order_by('-pub_date')
```

```
context = {
    'posts': all_posts, # 'posts' sẽ là tên biến trong template
    'page_title': "Danh sách bài viết blog"
}

return render(request, 'blog/post_list.html', context)
```

Trong `blog/post_list.html`, bạn có thể truy cập `{{ posts }}` và `{{ page_title }}`.

13.2. Thao tác trong Template & Static Files

Các thẻ điều khiển luồng trong Template: Vòng lặp (`{% for %}`), điều kiện (`{% if %}`)

- 1./ **Vòng lặp `{% for %}`:** Dùng để lặp qua một tập hợp dữ liệu (List, QuerySet, Dictionary, v.v.).

```
<ul>
    {% for post in posts %}
        <li>
            <h2>{{ post.title }}</h2>
            <p>{{ post.content|truncatechars:100 }}</p> {# Filter: cắt
bớt chuỗi #}
            <small>Đăng ngày: {{ post.pub_date|date:"F d, Y" }}</small>
{# Filter: định dạng ngày #}
        </li>
    {% empty %} {# Tùy chọn: hiển thị nếu tập hợp rỗng #}
        <li>Chưa có bài viết nào được tìm thấy.</li>
    {% endfor %}
</ul>
```

- `forloop.counter`: Số lần lặp hiện tại (bắt đầu từ 1).
- `forloop.counter0`: Số lần lặp hiện tại (bắt đầu từ 0).
- `forloop.first`: True nếu đây là lần lặp đầu tiên.
- `forloop.last`: True nếu đây là lần lặp cuối cùng.

- 2./ **Điều kiện `{% if %}`:** Dùng để hiển thị nội dung có điều kiện.

```
{% if user.is_staff %}
    <p>Chào mừng quản trị viên!</p>
{% elif user.is_authenticated %}
    <p>Chào mừng, {{ user.username }}!</p>
{% else %}
    <p>Vui lòng đăng nhập.</p>
{% endif %}
```

```
{% if item.price > 100 %}
    <p style="color: red;">Sản phẩm cao cấp!</p>
{% endif %}

{% if not items %} {# Kiểm tra nếu danh sách rỗng #}
    <p>Không có sản phẩm nào.</p>
{% endif %}
```

13.2.1. Kế thừa Template ({% extends %}, {% block %})

Kế thừa Template là một trong những tính năng mạnh mẽ nhất của DTL, giúp bạn tránh lặp lại mã HTML. Bạn định nghĩa một **base template** (template gốc) chứa cấu trúc chung của trang (ví dụ: header, footer, thanh điều hướng) và sau đó các template con sẽ "kế thừa" từ base template đó và chỉ ghi đè các phần cụ thể.

- **{% extends "base.html" %}**: Đặt ở đầu file template con để chỉ định nó kế thừa từ base.html.
- **{% block block_name %} ... {% endblock block_name %}**: Định nghĩa các "khối" nội dung trong base template mà template con có thể ghi đè.

Ví dụ base.html (trong thư mục templates cấp project):

```
<!DOCTYPE html>
<html lang="vi">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>{% block title %}My Django App{% endblock %}</title>
    <link rel="stylesheet" href="{% static 'css/style.css' %}"> {# Liên
kết với static file #}
</head>
<body>
    <header>
        <h1>{% block header_content %}Welcome{% endblock %}</h1>
        <nav>
            <a href="/">Trang chủ</a> |
            <a href="/blog/">Bài viết</a> |
            <a href="/admin/">Admin</a>
        </nav>
    </header>
```

```
<main>
    {% block content %}
        {%# Nội dung mặc định nếu không có template con nào ghi đè #}
    {% endblock %}
</main>

<footer>
    <p>&copy; 2025 My Django App</p>
</footer>

<script src="{% static 'js/main.js' %}"></script>
</body>
</html>
```

Ví dụ blog/post_list.html (kế thừa từ base.html):

```
{% extends "base.html" %} {%# Kế thừa từ base.html #}

{% block title %}Danh sách Bài viết - {{ block.super }}{% endblock %} {%#
Ghi đè title, sử dụng block.super để giữ nội dung gốc #}

{% block header_content %}Các bài viết mới nhất{% endblock %} {%# Ghi đè
nội dung header #}

{% block content %} {%# Ghi đè khối content #}
    <h2>Tất cả Bài viết</h2>
    <ul>
        {% for post in posts %}
            <li>
                <h3><a href="{% url 'blog:detail' post.id %}">{{
post.title }}</a></h3> {%# Sử dụng url tag #}
                <p>{{ post.content|truncatewords:30 }}</p>
                <small>Ngày đăng: {{ post.pub_date|date:"d/m/Y"
}}</small>
            </li>
            {% empty %}
                <li>Chưa có bài viết nào.</li>
            {% endfor %}
        </ul>
    {% endblock %}
```

- {% url 'blog:detail' post.id %}: Đây là thẻ url của Django. Nó giúp bạn tạo URL động dựa trên tên định danh (name) của URL pattern và các tham số. blog là app_name, detail là name của URL pattern.

13.2.2. Giới thiệu Static Files (CSS, JavaScript, hình ảnh) và cách phục vụ chúng trong Django

Static Files là các tệp không thay đổi nội dung (tĩnh) được phục vụ trực tiếp cho trình duyệt của người dùng. Chúng bao gồm:

- CSS (Cascading Style Sheets)
- JavaScript
- Hình ảnh (JPG, PNG, GIF)
- Phông chữ (Fonts)

Cách Django phục vụ Static Files:

1. Cấu hình trong settings.py:

- **STATIC_URL**: URL cơ sở để truy cập static files. Mặc định là '/static/'.
- **STATICFILES_DIRS**: (Tùy chọn) Một list các thư mục khác ngoài thư mục static/ của mỗi App mà Django nên tìm kiếm static files. Thường dùng để đặt các static files dùng chung cho toàn project.
- **STATIC_ROOT**: (Chỉ dùng cho production) Thư mục mà tất cả static files từ các App và STATICFILES_DIRS sẽ được thu thập vào khi triển khai (dùng lệnh python manage.py collectstatic).

```
# myportfolio/settings.py

STATIC_URL = '/static/'

# Thư mục cho các static files dùng chung toàn project
STATICFILES_DIRS = [
    os.path.join(BASE_DIR, 'static'),
]

# STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles') # Chỉ dùng trong
production
```

Bạn nên tạo thư mục myportfolio/static/ cho các static files chung.

2. **Đặt Static Files trong App:** Trong mỗi App, hãy tạo một thư mục static/ và bên trong đó là một thư mục con cùng tên với App của bạn để tránh xung đột tên.

```
blog/  
└── static/  
    └── blog/  
        ├── css/  
        │   └── style.css  
        ├── js/  
        │   └── script.js  
        └── img/  
            └── logo.png
```

- Để truy cập blog/static/blog/css/style.css, bạn sẽ dùng {% static 'blog/css/style.css' %}
3. **Sử dụng trong Template:** Để sử dụng static files trong template, bạn cần tải thẻ static và dùng cú pháp {% static 'path/to/your/file.css' %}.

```
{% load static %} {# Luôn đặt ở đầu file template #}  
  
<link rel="stylesheet" href="{% static 'blog/css/style.css' %}">  
  
<script src="{% static 'blog/js/script.js' %}"></script>
```

4. **Phục vụ Static Files trong phát triển:** Django development server tự động phục vụ static files.

13.3. Form cơ bản (HTML Forms) & Django Forms

13.3.1. Khái niệm HTML Forms: <form>, <input>, <button>

HTML Forms là các thành phần của trang web cho phép người dùng nhập liệu và gửi dữ liệu đó lên máy chủ.

- <form> tag:
 - Xác định một biểu mẫu HTML.
 - action: URL mà dữ liệu form sẽ được gửi đến khi submit.
 - method: Phương thức HTTP để gửi dữ liệu (GET hoặc POST).
- <input> tag:
 - Các trường nhập liệu khác nhau (text, password, checkbox, radio, submit, hidden, v.v.).

- **type:** Loại trường nhập liệu.
- **name:** Tên của trường, dùng để truy cập dữ liệu khi gửi lên server.
- **value:** Giá trị ban đầu của trường.
- **<textarea> tag:** Dùng cho các trường văn bản đa dòng.
- **<select> tag & <option> tag:** Dùng cho các dropdown list.
- **<button> tag:** Nút để gửi form hoặc thực hiện hành động khác.

Ví dụ HTML Form:

```
<form action="/submit-feedback/" method="post">
  <label for="name">Tên của bạn:</label><br>
  <input type="text" id="name" name="user_name" required><br><br>

  <label for="email">Email:</label><br>
  <input type="email" id="email" name="user_email"><br><br>

  <label for="message">Tin nhắn:</label><br>
  <textarea id="message" name="user_message" rows="5"
cols="40"></textarea><br><br>

  <button type="submit">Gửi phản hồi</button>
</form>
```

13.3.2. Phương thức HTTP: GET và POST

Khi gửi dữ liệu từ Form, hai phương thức HTTP phổ biến nhất là GET và POST:

- **GET method:**
 - Gửi dữ liệu form dưới dạng tham số trong URL (query string).
 - Ví dụ: `yourdomain.com/search?query=django&category=web`
 - **Thích hợp cho:** Các yêu cầu truy vấn dữ liệu (tìm kiếm, lọc), nơi dữ liệu không nhạy cảm và có thể được bookmark.
 - **Hạn chế:** Giới hạn kích thước dữ liệu, dữ liệu hiển thị trên URL (không an toàn cho mật khẩu).
- **POST method:**
 - Gửi dữ liệu form trong phần **body** của yêu cầu HTTP.
 - Dữ liệu không hiển thị trên URL.

- **Thích hợp cho:** Gửi dữ liệu nhạy cảm (mật khẩu), tạo hoặc cập nhật tài nguyên trên server, gửi lượng lớn dữ liệu.
- **Quan trọng:** Django yêu cầu bạn sử dụng thẻ {% csrf_token %} bên trong mọi form POST để bảo vệ khỏi tấn công CSRF (Cross-Site Request Forgery).

13.3.3. Cách Django nhận và xử lý dữ liệu từ Form (truy cập request.POST)

Khi một form được submit với phương thức POST, Django sẽ đưa dữ liệu form vào thuộc tính request.POST của đối tượng HttpRequest. Đây là một Dictionary-like object (thực tế là QueryDict).

```
# blog/views.py

from django.shortcuts import render, redirect # Import redirect

def create_post(request):
    if request.method == 'POST':
        # Dữ liệu từ form POST có trong request.POST
        title = request.POST.get('title') # 'title' là thuộc tính 'name'
        của input
        content = request.POST.get('content')

        if title and content:
            # Ví dụ: Lưu vào database (không dùng Django Forms API ở đây)
            # from .models import Post
            # Post.objects.create(title=title, content=content)
            # print(f"Đã nhận bài viết: {title} - {content}")
            return redirect('blog:list') # Chuyển hướng sau khi xử lý
            thành công POST
        else:
            print("Tiêu đề và nội dung không được trống.")
            # Hiển thị lại form với lỗi hoặc thông báo
    # Nếu là yêu cầu GET, hiển thị form trống
    return render(request, 'blog/post_form.html')
```

- request.POST.get('field_name'): Cách an toàn để lấy giá trị từ form, trả về None nếu trường không tồn tại.
- **redirect()**: Sau khi xử lý yêu cầu POST thành công (ví dụ: lưu dữ liệu vào database), bạn nên **chuyển hướng (redirect)** người dùng đến một trang khác (thường là trang hiển thị danh sách hoặc trang chi tiết). Điều này ngăn chặn việc gửi lại form khi người dùng làm mới trang.

13.3.4. Giới thiệu về Django Forms API

Việc xử lý form thủ công như trên rất dễ bị lỗi (ví dụ: thiếu kiểm tra dữ liệu đầu vào, không xử lý bảo mật). **Django Forms API** cung cấp một cách mạnh mẽ và an toàn để xử lý form.

Lợi ích của Django Forms:

- **Tạo form HTML dễ dàng:** Tự động render các trường form.
- **Kiểm tra tính hợp lệ (Validation):** Tự động kiểm tra các kiểu dữ liệu, độ dài, định dạng email/URL, v.v.
- **Xử lý lỗi:** Hiển thị thông báo lỗi cho người dùng một cách dễ dàng.
- **Tái sử dụng:** Có thể sử dụng lại cùng một định nghĩa form cho cả việc hiển thị và xử lý dữ liệu.
- **Bảo mật:** Tự động tích hợp CSRF token.

Các bước cơ bản của Django Forms API:

1. **Định nghĩa Form:** Tạo một lớp Form bằng cách kế thừa từ `forms.Form` hoặc `forms.ModelForm` (nếu form ánh xạ trực tiếp đến một Model).
- Thường được đặt trong file `forms.py` của App.

```
# blog/forms.py (Tạo file này nếu chưa có)
from django import forms
from .models import Post # Import Model Post

class PostForm(forms.ModelForm): # Kế thừa từ ModelForm nếu liên quan đến Model
    class Meta:
        model = Post # Chỉ định Model này
        fields = ['title', 'content'] # Các trường muốn hiển thị trong form
        widgets = {
            'title': forms.TextInput(attrs={'class': 'form-control',
            'placeholder': 'Tiêu đề bài viết'}),
            'content': forms.Textarea(attrs={'class': 'form-control',
            'rows': 5, 'placeholder': 'Nội dung bài viết'})
        }
        labels = { # Tùy chỉnh nhãn
            'title': 'Tiêu đề',
            'content': 'Nội dung'
        }
```

```
    }
    help_texts = { # Gợi ý
        'title': 'Tiêu đề không quá 200 ký tự'
    }

# Hoặc nếu không liên quan đến Model:
# class ContactForm(forms.Form):
#     name = forms.CharField(max_length=100)
#     email = forms.EmailField()
#     message = forms.CharField(widget=forms.Textarea)
```

2. **Render Form trong Template:** Trong View, tạo một instance của Form và truyền nó vào context.

```
# blog/views.py
from .forms import PostForm

def create_post(request):
    form = PostForm() # Tạo một instance của form
    return render(request, 'blog/post_form.html', {'form': form})

Trong Template:
HTML
<form method="post">
    {% csrf_token %} {# Bắt buộc với form POST #}
    {{ form.as_p }} {# Render form dưới dạng các đoạn văn (p tags) #}
    <button type="submit">Gửi</button>
</form>
```

3. Xử lý dữ liệu Form trong View:

```
# blog/views.py
from django.shortcuts import render, redirect
from .forms import PostForm # Import Form của bạn

def create_post(request):
    if request.method == 'POST':
        form = PostForm(request.POST) # Tạo form với dữ liệu từ
        request.POST

        if form.is_valid(): # Kiểm tra tính hợp lệ của dữ liệu
            form.save() # Lưu dữ liệu vào database (chỉ dùng với
            ModelForm)

            # Hoặc form.cleaned_data để truy cập dữ liệu đã được validate
```

```
        # title = form.cleaned_data['title']
        # content = form.cleaned_data['content']
        # Post.objects.create(title=title, content=content, ...)

        return redirect('blog:list') # Chuyển hướng sau khi thành
công
    else: # Nếu là GET request
        form = PostForm() # Tạo form trống

    return render(request, 'blog/post_form.html', {'form': form})
```

- `form = PostForm(request.POST)`: Khởi tạo form với dữ liệu được gửi từ request POST.
- `form.is_valid()`: Phương thức này chạy tất cả các quy tắc kiểm tra hợp lệ được định nghĩa trong Form (hoặc Model của nó). Trả về True nếu tất cả dữ liệu hợp lệ, False nếu không.
- `form.save()`: (Chỉ áp dụng cho `ModelForm`) Phương thức này sẽ tạo hoặc cập nhật một đối tượng Model trong database với dữ liệu đã được validate.
- `form.cleaned_data`: Một Dictionary chứa dữ liệu đã được validate và chuyển đổi sang đúng kiểu Python.

BÀI TẬP:

1. Sử dụng Template Inheritance (kế thừa template) để tạo file `base.html` và các template khác kế thừa layout.
2. Tích hợp Bootstrap (hoặc CSS đơn giản) thông qua Static Files. Tạo giao diện form đẹp hơn cho việc thêm bài viết.
3. Xây dựng View và Template cho chức năng chỉnh sửa bài viết (`PostUpdateView` - CBV). Yêu cầu người dùng phải nhập lại ID bài viết để xác nhận chỉnh sửa.

THỰC HÀNH 13: TEMPLATES & FORMS

13.1. Hiển thị dữ liệu bằng Template

13.1.1. Tạo file HTML template

Mục tiêu: Chuẩn bị các file Template để sử dụng.

Các bước thực hành:

1. Đảm bảo bạn đã cấu hình TEMPLATES trong settings.py để tìm kiếm template trong APP_DIRS và có thể cả thư mục templates cấp project.
2. Trong thư mục blog/, tạo một thư mục con templates/ (nếu chưa có).
3. Bên trong blog/templates/, tạo thêm một thư mục con nữa tên là blog/ (để tránh xung đột tên).
4. Cấu trúc sẽ là: myportfolio/blog/templates/blog/
5. Trong myportfolio/blog/templates/blog/, tạo file post_list.html.

```
<!DOCTYPE html>
<html lang="vi">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Danh sách Bài viết</title>
</head>
<body>
    <h1>Tất cả Bài viết</h1>
    <ul>
        {% for post in posts %}
            <li>
                <h2>{{ post.title }}</h2>
                <p>{{ post.content|truncatewords:50 }}</p> {# Cắt ngắn
nội dung còn 50 từ #}
                <small>Đăng ngày: {{ post.pub_date|date:"d/m/Y H:i"
}}</small>
            </li>
            {% empty %}
                <li>Chưa có bài viết nào được tìm thấy.</li>
            {% endfor %}
        </ul>
```

```
</body>
</html>
```

13.1.2. Sửa View để render template và truyền danh sách các đối tượng Model vào template

Mục tiêu: Kết nối View với Template và truyền dữ liệu.

Các bước thực hành:

1. Mở file blog/views.py.
2. Sửa lại View post_list để sử dụng render():

```
# blog/views.py
from django.shortcuts import render # Đảm bảo đã import render
from .models import Post

# ... (giữ lại các views khác nếu cần, hoặc xóa nếu không dùng)

def post_list(request):
    """
    View để hiển thị danh sách tất cả các bài viết sử dụng template.
    """
    all_posts = Post.objects.order_by('-pub_date')
    context = {
        'posts': all_posts, # Truyền QuerySet vào context
    }
    return render(request, 'blog/post_list.html', context)
```

3. Lưu file views.py.

13.1.3. Hiển thị danh sách các đối tượng (bài viết/sản phẩm) bằng vòng lặp trong template

Mục tiêu: Kiểm tra kết quả hiển thị của Template.

Các bước thực hành:

1. Đảm bảo bạn đã có ít nhất một vài bài viết trong database (tạo qua Django Admin hoặc shell).
2. Chạy máy chủ phát triển: python manage.py runserver.
3. Truy cập URL đã được ánh xạ đến post_list View của bạn (ví dụ: <http://127.0.0.1:8000/blog/>).

4. Bạn sẽ thấy danh sách các bài viết được hiển thị với tiêu đề, nội dung (đã cắt ngắn) và ngày đăng.

13.2. Kế thừa Template & Static Files

13.2.1. Tạo một base template và các template con kế thừa

Mục tiêu: Tổ chức cấu trúc Template theo hướng DRY.

Các bước thực hành:

1. Tạo thư mục templates cấp project:
 - Trong thư mục gốc của project myportfolio, tạo một thư mục mới tên là templates.
 - Cấu trúc: myportfolio/templates/
2. Tạo base.html trong thư mục templates cấp project:
 - Nội dung base.html:

```
{% load static %} {# Tải static tag để dùng cho CSS/JS #}
<!DOCTYPE html>
<html lang="vi">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>{% block title %}Ứng dụng Django của tôi{% endblock %}</title>
    <link rel="stylesheet" href="{% static 'css/style.css' %}"> {# Sẽ tạo
file này sau #}
</head>
<body>
    <header>
        <nav>
            <a href="{% url 'blog:home' %}">Trang chủ</a> |
            <a href="{% url 'blog:list' %}">Bài viết</a> |
            <a href="{% url 'admin:index' %}">Admin</a>
        </nav>
    </header>

    <main>
        {% block content %}
            {# Nội dung chính của trang sẽ ở đây #}
        {% endblock %}
```

```
</main>

<footer>
    <p>&copy; 2025 Blog Django - Mọi quyền được bảo lưu.</p>
</footer>

<script src="{% static 'js/main.js' %}"></script> {# Sẽ tạo file này
sau #}
</body>
</html>
```

3. Sửa post_list.html để kế thừa base.html:

- Mở myportfolio/blog/templates/blog/post_list.html.
- Thay đổi toàn bộ nội dung của nó như sau:

```
{% extends "base.html" %} {# Kế thừa từ base.html #}

{% block title %}Danh sách Bài viết - {% endblock %} {# Ghi đề title #}

{% block content %} {# Ghi đề khối content #}
    <h1>Tất cả Bài viết</h1>
    <ul>
        {% for post in posts %}
            <li>
                <h3><a href="{% url 'blog:detail' post.id %}">{{
post.title }}</a></h3> {# Sẽ cần thêm detail URL ở tiết sau #}
                <p>{{ post.content|truncatewords:50 }}</p>
                <small>Ngày đăng: {{ post.pub_date|date:"d/m/Y H:i"
}}</small>
            </li>
            {% empty %}
                <li>Chưa có bài viết nào được tìm thấy.</li>
            {% endfor %}
        </ul>
    {% endblock %}
```

- Đảm bảo bạn đã thêm URL pattern cho post_detail trong blog/urls.py để liên kết hoạt động.

13.2.2. Sử dụng các thẻ điều kiện trong template

Mục tiêu: Thêm logic hiển thị có điều kiện.

Các bước thực hành:

1. Sửa `blog/post_list.html` để thêm một điều kiện đơn giản (ví dụ: hiển thị một thông báo nếu bài viết mới hơn 24 giờ). Để làm điều này, bạn cần thêm một phương thức vào Model `Post` để kiểm tra thời gian.

- Mở `blog/models.py` và thêm phương thức `was_published_recently()`:

```
# blog/models.py
from django.db import models
import datetime # Import datetime
from django.utils import timezone # Import timezone

class Post(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()
    pub_date = models.DateTimeField('date published')

    def __str__(self):
        return self.title

    def was_published_recently(self):
        now = timezone.now()
        return now - datetime.timedelta(days=1) <= self.pub_date <= now
    was_published_recently.admin_order_field = 'pub_date'
    was_published_recently.boolean = True
    was_published_recently.short_description = 'Published recently?'

    class Meta:
        ordering = ['-pub_date']
```

- Sửa `blog/post_list.html`:

```
{% extends "base.html" %}

{% block title %}Danh sách Bài viết - {% endblock %}

{% block content %}
    <h1>Tất cả Bài viết</h1>
    <ul>
        {% for post in posts %}
            <li>
                <h3><a href="{% url 'blog:detail' post.id %}">{{
post.title }}</a></h3>
```

```

        <p>{{ post.content|truncatewords:50 }}</p>
        <small>Ngày đăng: {{ post.pub_date|date:"d/m/Y H:i"
    }}</small>

        {% if post.was_published_recently %} {% # Sử dụng điều kiện
#}

            <span style="color: green; font-weight:
bold;">(MỚI!)</span>
            {% endif %}

        </li>
        {% empty %}
            <li>Chưa có bài viết nào được tìm thấy.</li>
        {% endfor %}

    </ul>
{% endblock %}
```

2. Lưu các file và kiểm tra trên trình duyệt.

13.2.3. Thiết lập static files và thêm CSS/JavaScript đơn giản vào template

Mục tiêu: Trang trí và thêm tương tác cho website.

Các bước thực hành:

1. Cấu hình STATICFILES_DIRS trong settings.py:

- Mở myportfolio/settings.py.
- Đảm bảo dòng import os ở đầu file.
- Thêm/sửa STATIC_URL và STATICFILES_DIRS:

```
# myportfolio/settings.py

STATIC_URL = '/static/'
STATICFILES_DIRS = [
    os.path.join(BASE_DIR, 'static'),
]
# STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles') # Bỏ comment nếu
bạn triển khai lên production
```

2. Tạo thư mục static cấp project và các file CSS/JS:

- Trong thư mục gốc của project myportfolio, tạo một thư mục mới tên là static.
- Bên trong myportfolio/static/, tạo các thư mục con css/ và js/.
- Tạo file style.css trong myportfolio/static/css/:

```
/* myportfolio/static/css/style.css */
body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
    background-color: #f4f4f4;
    color: #333;
}
header {
    background-color: #333;
    color: #fff;
    padding: 1rem 0;
    text-align: center;
}
nav a {
    color: #fff;
    text-decoration: none;
    margin: 0 10px;
}
main {
    padding: 20px;
    max-width: 800px;
    margin: 20px auto;
    background-color: #fff;
    border-radius: 8px;
    box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}
ul {
    list-style: none;
    padding: 0;
}
li {
    background-color: #e9e9e9;
    margin-bottom: 10px;
    padding: 15px;
    border-radius: 5px;
}
footer {
    text-align: center;
    padding: 1rem 0;
```

```
background-color: #333;
color: #fff;
position: fixed;
bottom: 0;
width: 100%;
}
```

- Tạo file main.js trong myportfolio/static/js/:

```
// myportfolio/static/js/main.js
console.log("JavaScript từ static files đã được tải!");
document.addEventListener('DOMContentLoaded', () => {
    const pageTitle = document.querySelector('h1');
    if (pageTitle) {
        pageTitle.style.color = '#0056b3';
    }
});
```

3. Thêm các file này vào base.html:

- Mở myportfolio/templates/base.html.
- Đảm bảo dòng {% load static %} ở đầu file.
- Đảm bảo các thẻ <link> và <script> đã được thêm như trong ví dụ base.html ở phần lý thuyết.

4. Lưu tất cả các file.

5. Chạy python manage.py runserver và truy cập lại trang web. Bạn sẽ thấy các styles và script của bạn được áp dụng. Kiểm tra console của trình duyệt để xem thông báo từ JavaScript.

13.3. Xử lý Form đơn giản (Django Forms)

13.3.1. Tạo một Django Form đơn giản (ví dụ: Form nhập liệu Post/Item)

Mục tiêu: Định nghĩa cấu trúc Form bằng Django Forms API.

Các bước thực hành:

1. Tạo file blog/forms.py:

```
# blog/forms.py
from django import forms
from .models import Post # Import Model Post
```

```
class PostForm(forms.ModelForm):
    class Meta:
        model = Post
        fields = ['title', 'content'] # Các trường sẽ xuất hiện trong form
        labels = {
            'title': 'Tiêu đề bài viết',
            'content': 'Nội dung'
        }
        widgets = { # Tùy chỉnh HTML widget cho các trường
            'title': forms.TextInput(attrs={'class': 'form-control',
            'placeholder': 'Nhập tiêu đề...'}),
            'content': forms.Textarea(attrs={'class': 'form-control',
            'rows': 8, 'placeholder': 'Viết nội dung bài viết...'})
        }
```

2. Lưu file forms.py.

13.3.2. Render Form trong template

Mục tiêu: Hiển thị form đã định nghĩa trong một template HTML.

Các bước thực hành:

1. Tạo View để hiển thị Form:
 - Mở blog/views.py.
 - Import PostForm và thêm View create_post:

```
# blog/views.py
from django.shortcuts import render, redirect
from .models import Post
from .forms import PostForm # Import PostForm

# ... (các views khác) ...

def create_post(request):
    form = PostForm() # Tạo một instance form trống cho GET request
    return render(request, 'blog/post_form.html', {'form': form})
```

2. Tạo URL pattern cho View create_post:
 - Mở blog/urls.py.
 - Thêm dòng sau vào urlpatterns:

```
# blog/urls.py
from django.urls import path
from . import views

app_name = 'blog'

urlpatterns = [
    path('home/', views.home_page, name='home'),
    path('', views.post_list, name='list'),
    path('<int:post_id>/', views.post_detail, name='detail'),
    path('create/', views.create_post, name='create'), # URL để tạo bài
viết mới
]
```

3. Tạo template post_form.html:

- Trong myportfolio/blog/templates/blog/, tạo file post_form.html.
- Nội dung post_form.html:

```
{% extends "base.html" %}
{% load static %}

{% block title %}Tạo Bài viết Mới - {% endblock %}

{% block content %}
    <h1>Tạo Bài viết Mới</h1>
    <form method="post">
        {% csrf_token %} {% # LUÔN LUÔN CÓ CHO FORM POST #}

        {% # Render form dưới dạng các đoạn văn (p tags) #}
        {{ form.as_p }}

        {% # Hoặc render từng trường thủ công để tùy chỉnh nhiều hơn #}
        <button type="submit" class="btn btn-primary">Luu Bài
viết</button>
    </form>
{% endblock %}
```

- {{ form.as_p }} là cách nhanh nhất để render form. Django sẽ tự động tạo thẻ <p> cho mỗi trường.

- Phần comment là ví dụ về cách bạn có thể render từng trường một cách thủ công để tùy chỉnh CSS/HTML.

4. Lưu tất cả các file.

5. Chạy python manage.py runserver và truy cập <http://127.0.0.1:8000/blog/create/>. Bạn sẽ thấy form được hiển thị.

13.3.3. Viết View để nhận và xử lý dữ liệu POST từ Form, kiểm tra tính hợp lệ (form.is_valid()) và lưu vào database

Mục tiêu: Hoàn thiện luồng xử lý form: nhận dữ liệu, validate, lưu vào DB và chuyển hướng.

Các bước thực hành:

1. Mở blog/views.py.
2. Sửa lại View create_post để xử lý cả GET và POST request:

```
# blog/views.py
from django.shortcuts import render, redirect, get_object_or_404 # Import
get_object_or_404
from .models import Post
from .forms import PostForm

def post_list(request):
    all_posts = Post.objects.order_by('-pub_date')
    context = {
        'posts': all_posts,
    }
    return render(request, 'blog/post_list.html', context)

def post_detail(request, post_id):
    # Dùng get_object_or_404 thay cho try-except Http404 thủ công
    post = get_object_or_404(Post, pk=post_id)
    return render(request, 'blog/post_detail.html', {'post': post}) # Sẽ
    tạo template này sau

def create_post(request):
    if request.method == 'POST':
        form = PostForm(request.POST) # Bind dữ liệu POST vào form
        if form.is_valid(): # Kiểm tra tính hợp lệ
```

```
        post = form.save(commit=False) # Lấy đối tượng Post nhưng
chưa lưu vào DB
        # Nếu bạn muốn thêm các trường khác không có trong form (ví
dụ: tác giả, tự động set pub_date)
        # post.author = request.user # Nếu bạn có hệ thống người dùng
        post.pub_date = timezone.now() # Cần import timezone nếu
không dùng auto_now_add=True trong Model
        post.save() # Lưu đối tượng vào DB
        print(f"Đã tạo bài viết: {post.title}")
        return redirect('blog:list') # Chuyển hướng đến trang danh
sách bài viết
    else:
        # Form không hợp lệ, render lại form với các lỗi
        print("Form không hợp lệ:", form.errors) # In lỗi ra console
để debug
    else: # Nếu là GET request
        form = PostForm() # Tạo form trống

    return render(request, 'blog/post_form.html', {'form': form})
```

- `get_object_or_404(Model, **kwargs)`: Một hàm tiện ích của Django để lấy một đối tượng hoặc raise `Http404` nếu không tìm thấy. Rất tiện lợi.
- `form.save(commit=False)`: Với `ModelForm`, phương thức `save()` sẽ tạo hoặc cập nhật một đối tượng `Model`. `commit=False` nghĩa là nó sẽ tạo đối tượng nhưng chưa lưu vào database, cho phép bạn thêm các giá trị khác trước khi gọi `.save()` lần nữa.

3. Lưu file `views.py`.

4. Chạy `python manage.py runserver` và truy cập `http://127.0.0.1:8000/blog/create/`.

5. Thử gửi form:

- Với dữ liệu hợp lệ: Bạn sẽ thấy bài viết được tạo và bạn được chuyển hướng về trang danh sách.
- Với dữ liệu không hợp lệ (ví dụ: để trống trường `title` nếu `CharField` không cho phép `blank=True`): Form sẽ hiển thị lại với các thông báo lỗi.

13.3.4. Để làm cho trang chi tiết (`post_detail`) hoạt động đúng cách:

1. Tạo file `post_detail.html`:

- Trong `myportfolio/blog/templates/blog/`, tạo file `post_detail.html`.
- Nội dung mẫu:

```
{% extends "base.html" %}

{% block title %}{{ post.title }} - {% endblock %}

{% block content %}
    <h1>{{ post.title }}</h1>
    <p><em>Đăng ngày: {{ post.pub_date|date:"d/m/Y H:i" }}</em></p>
    <div>
        {{ post.content|linebreaksbr }} {# filter linebreaksbr để chuyển
đổi newline thành <br> #}
    </div>
    <p><a href="{% url 'blog:list' %}">Quay lại danh sách bài
viết</a></p>
{% endblock %}
```

2. Lưu file post_detail.html.

3. Bạn có thể sửa lại liên kết trong post_list.html để trở đến trang chi tiết:

```
<h3><a href="{% url 'blog:detail' post.id %}">{{ post.title }}</a></h3>
```

4. Bây giờ, khi bạn nhấp vào tiêu đề bài viết trong danh sách, bạn sẽ được đưa đến trang chi tiết.